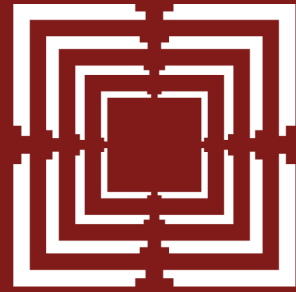


**Ahmedabad
University**

ENR497: B.Tech. Engg. Project

School of Engineering & Applied Science

(Winter 2026)



Ahmedabad
University

Agentic Cloud Cluster

Final Project Presentation

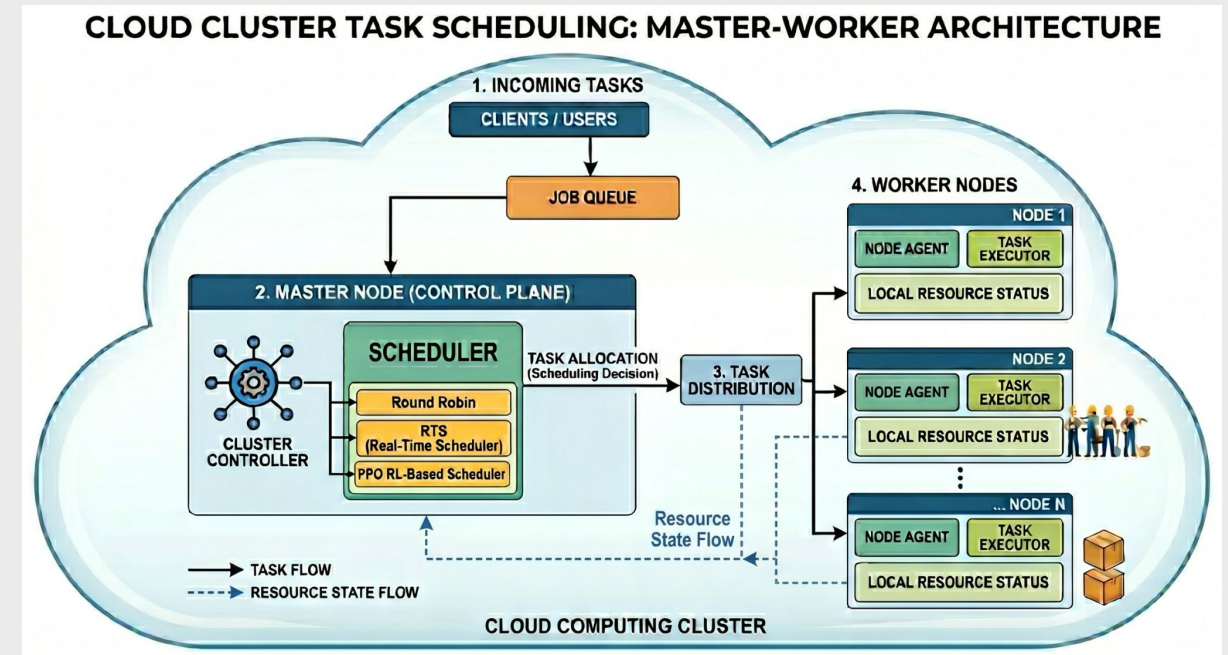
Sarthak Siddhpura
Mentor: Professor Sanjay Chaudhary
Jan 2026 - April 2026

Outline of Presentation

- Motivation
- Literature survey
- Overall objectives of the project
- Main outcomes of the project
- Methodology
- Progress and final results
- Novelty in the project
- Gantt chart for the project
- Who will benefit from your work ?

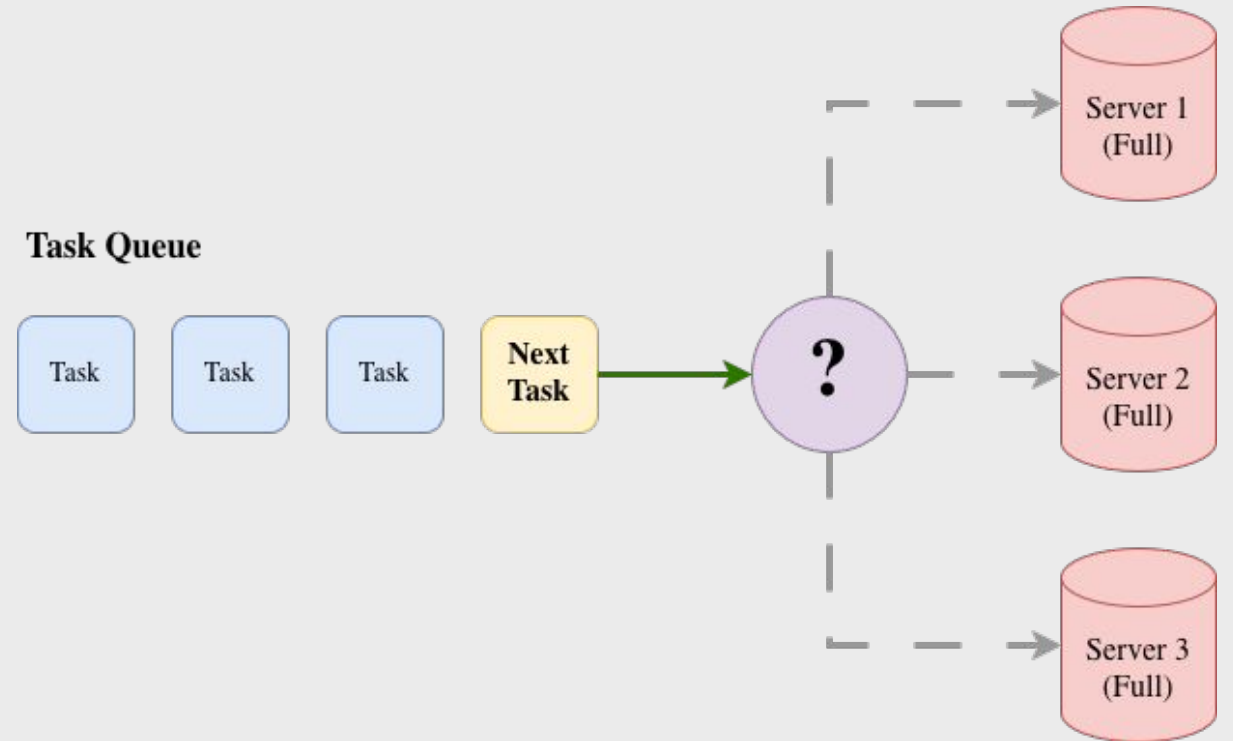
Agentic Cloud Cluster

- A distributed task execution system built in Go.
- Master-worker architecture coordinating Docker-based jobs.
- Task placement decisions enhanced with Proximal Policy Optimization (PPO).
- Clean separation between worker execution logic and the scheduling algorithm.



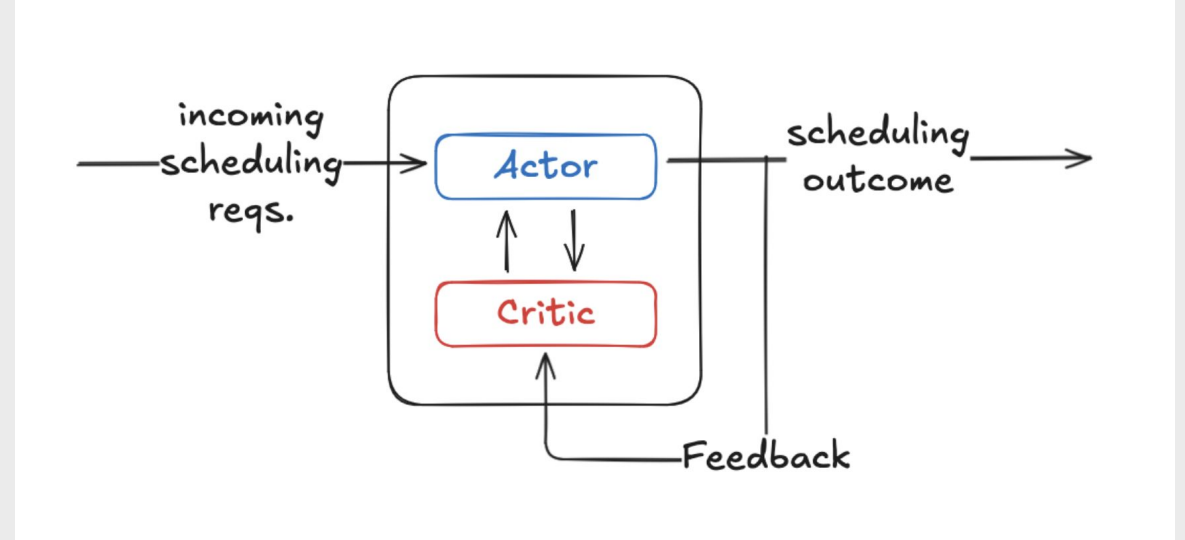
Motivation

- Simple scheduling rules (e.g., Round-Robin) fail to account for worker capacity or task urgency.
- Heuristic rules are hard to tune for mixed workloads (CPU-heavy vs. memory-heavy).
- Heterogeneous workers mean the "optimal" choice changes dynamically every few seconds.



Why Reinforcement Learning?

- **Scheduling** is a repeated decision problem modifying future cluster state.
- **State:** Current task requirements and worker resources.
- **Action:** The selected worker node.
- **Reward:** Feedback on placement feasibility, delay risk, and cluster balance.



Literature survey

- <Survey of similar work already existing (done by others industry/researchers)>

Evolution of Agentic Scheduling (Q-Tables to PPO)

- **Q-Tables:** Unsuitable due to massive, continuous state spaces in cluster environments.
- **Deep Q-Networks (DQN):** Difficult to handle dynamic action sets and masking invalid workers.
- **Proximal Policy Optimization (PPO):** Directly learns a policy, supports action masks, and uses stable clipped updates.

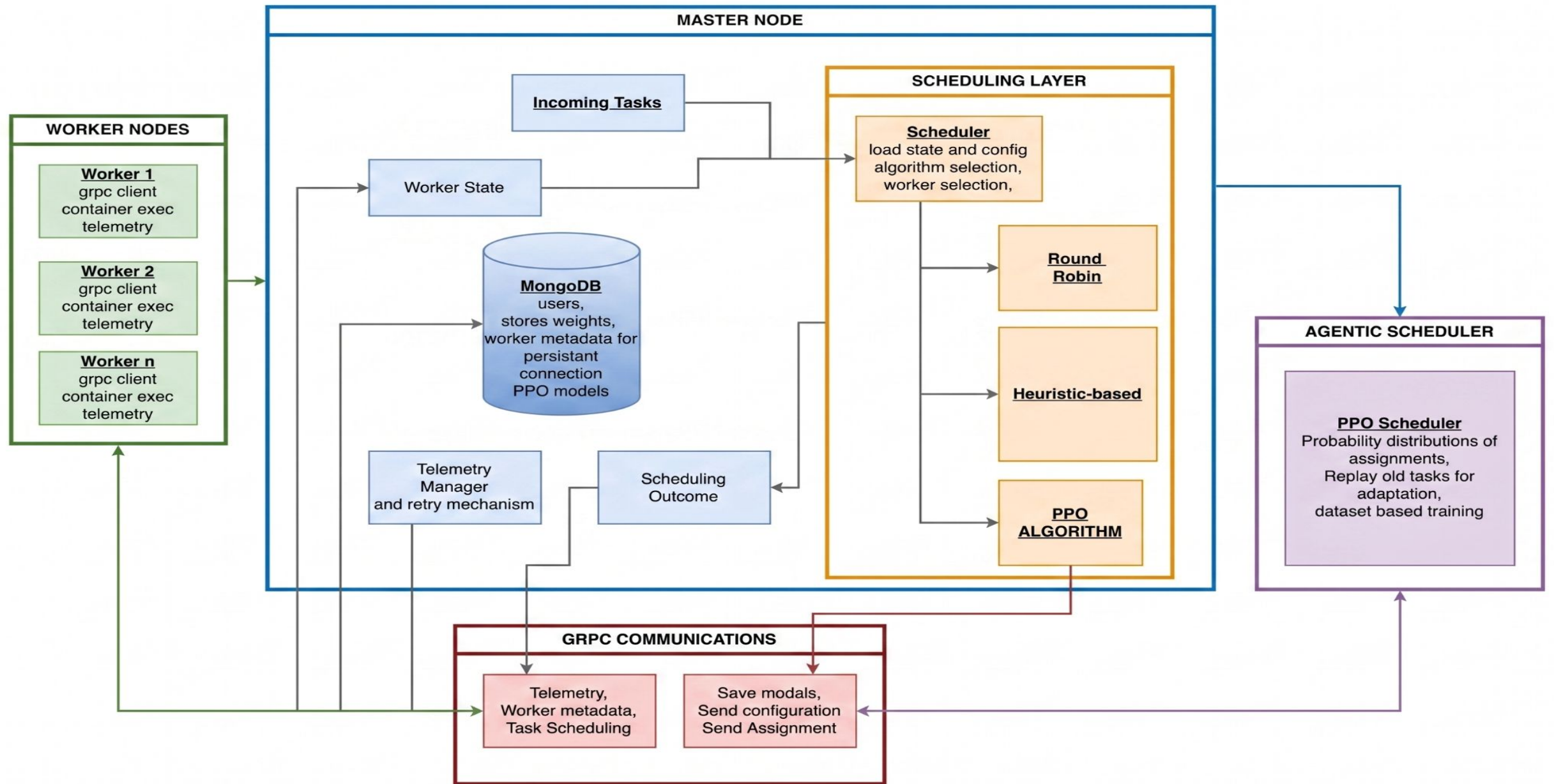
Relevant Tools and Technologies

- **Go:** Powers the highly concurrent master-worker control plane.
- **Docker:** Provides isolated, repeatable container execution environments.
- **gRPC & Protocol Buffers:** Ensures rigid, typed, and high-performance communication.
- **MongoDB:** Acts as the persistent storage layer for tasks, workers, and metadata.
- **Python/PyTorch:** Drives the decoupled PPO scheduling service.

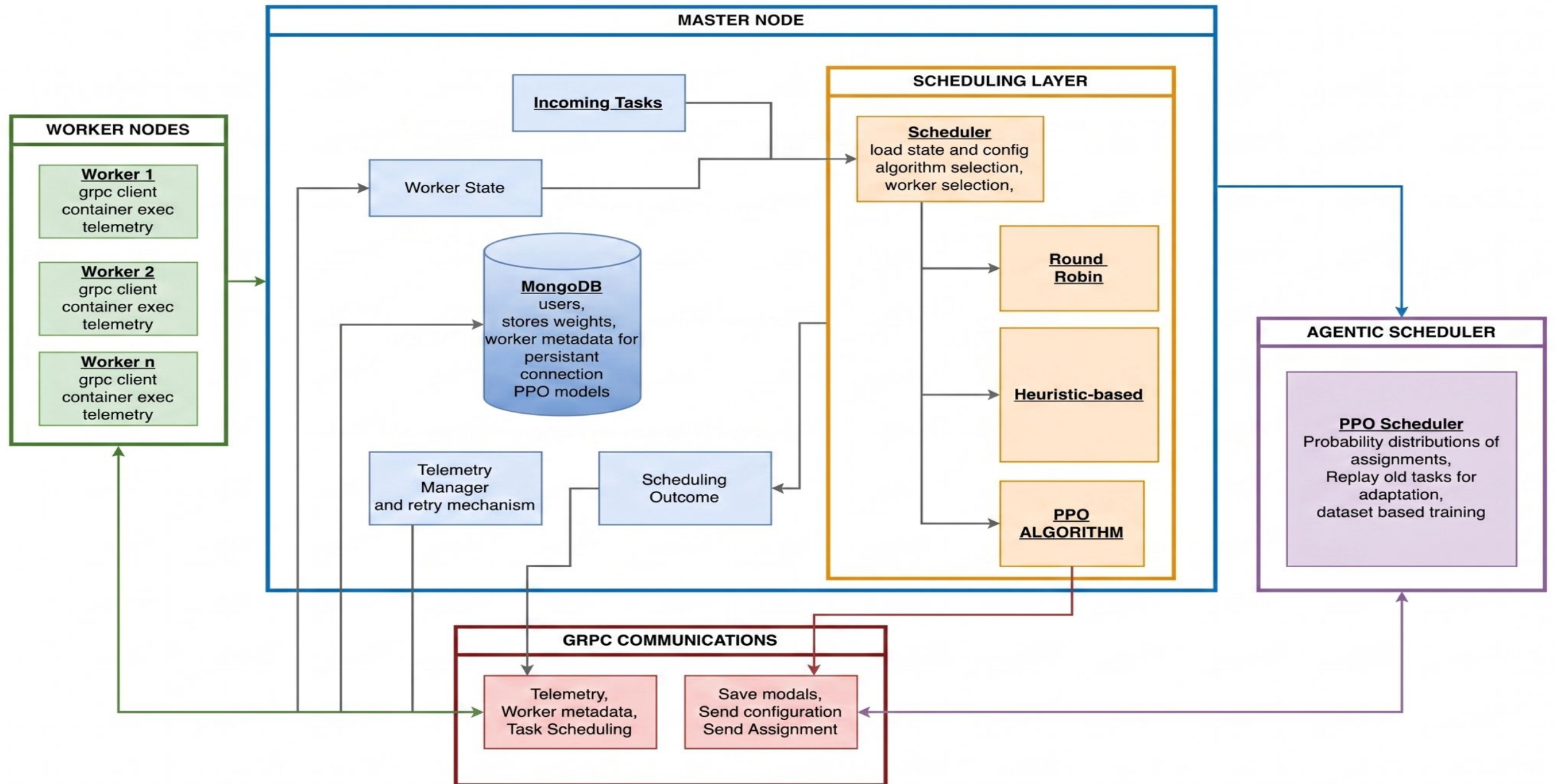
Objectives of the project

- Build a functional Go master-worker execution cluster.
- Implement worker registration, heartbeats, and resource tracking.
- Create a clean scheduling interface supporting both baselines and PPO.
- Train the PPO agent on realistic cluster traces with a comprehensive reward function.
- Conduct end-to-end benchmark campaigns to analyze system behavior.

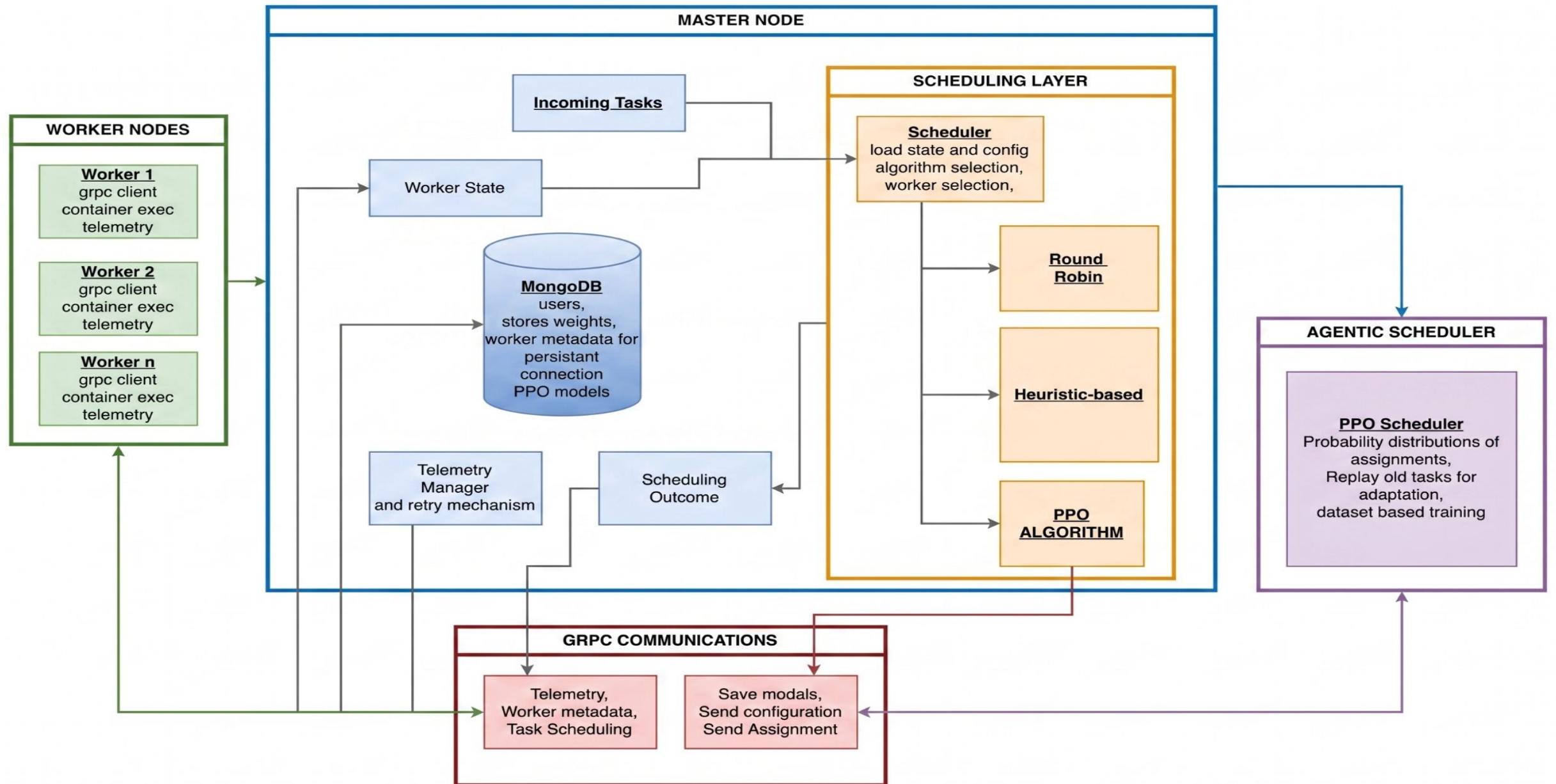
High-Level Architecture



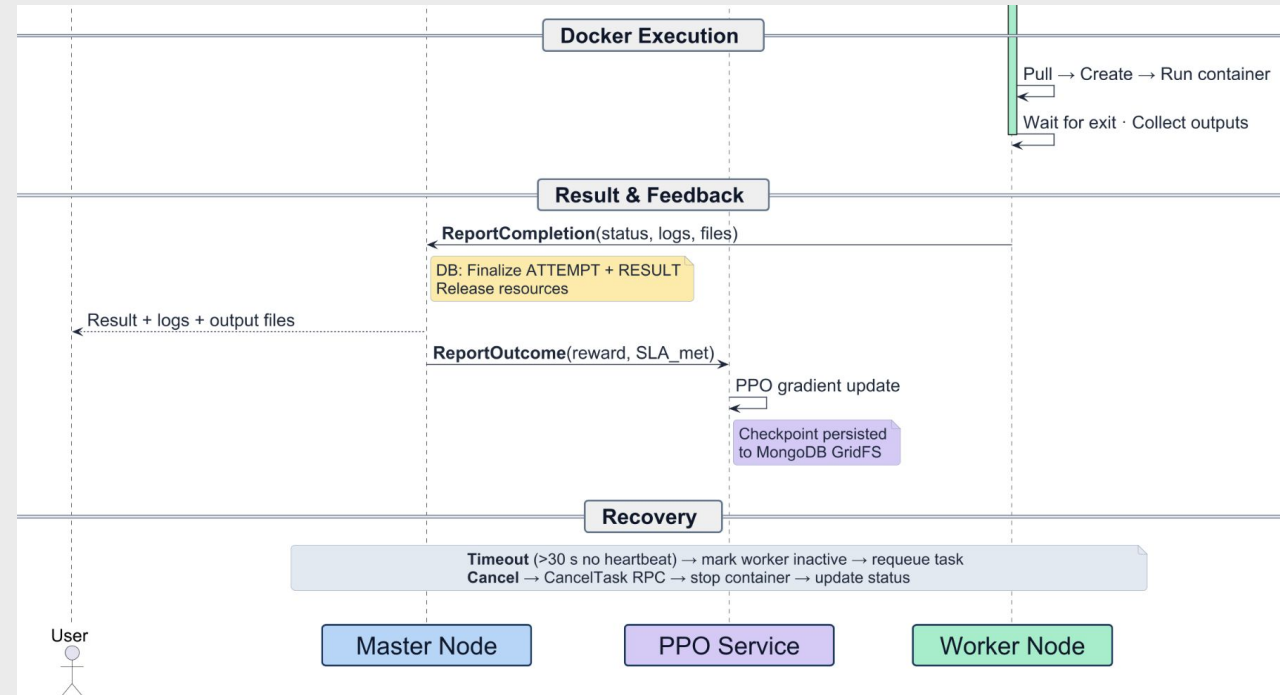
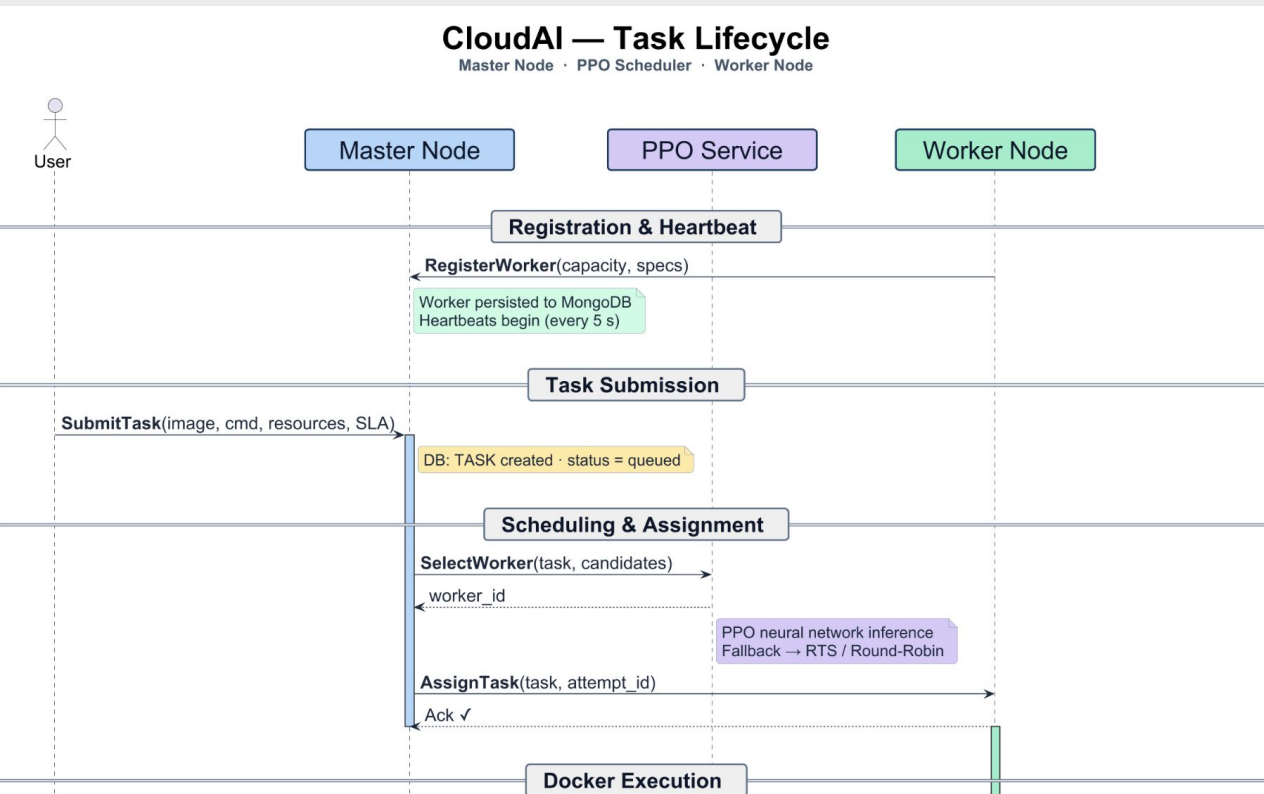
High-Level Architecture: Master Node



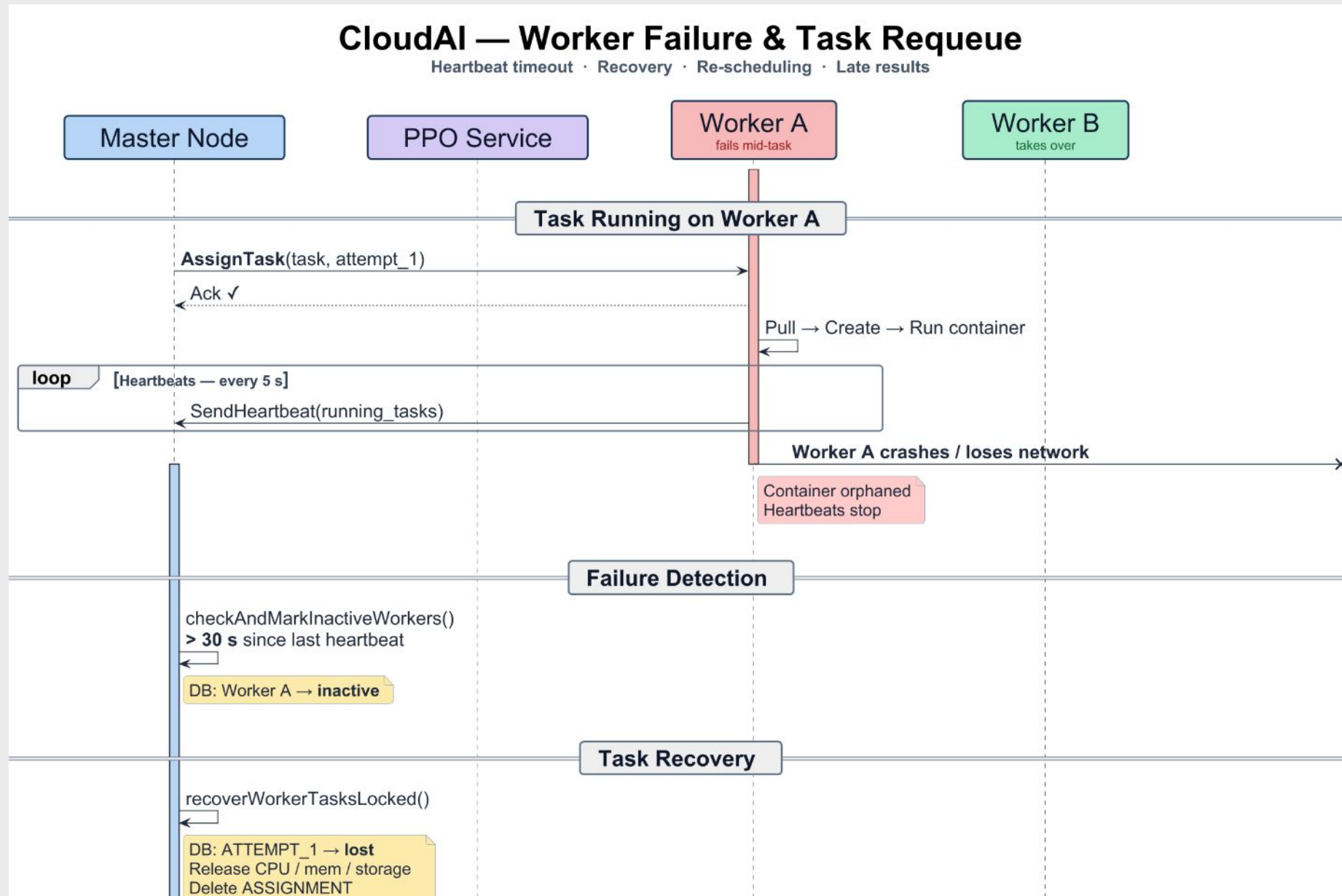
High-Level Architecture: Worker Nodes



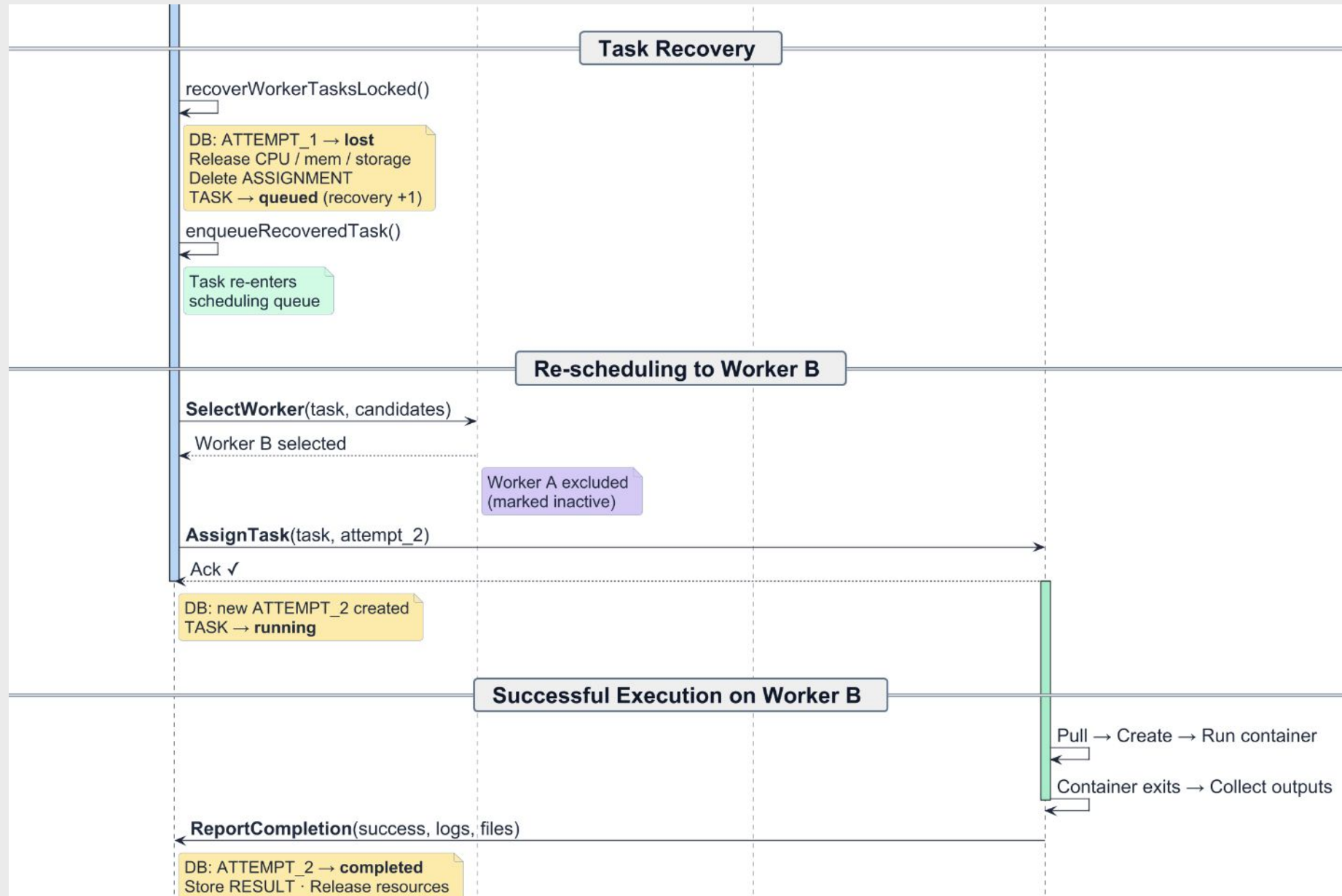
Task LifeCycle - Ideal



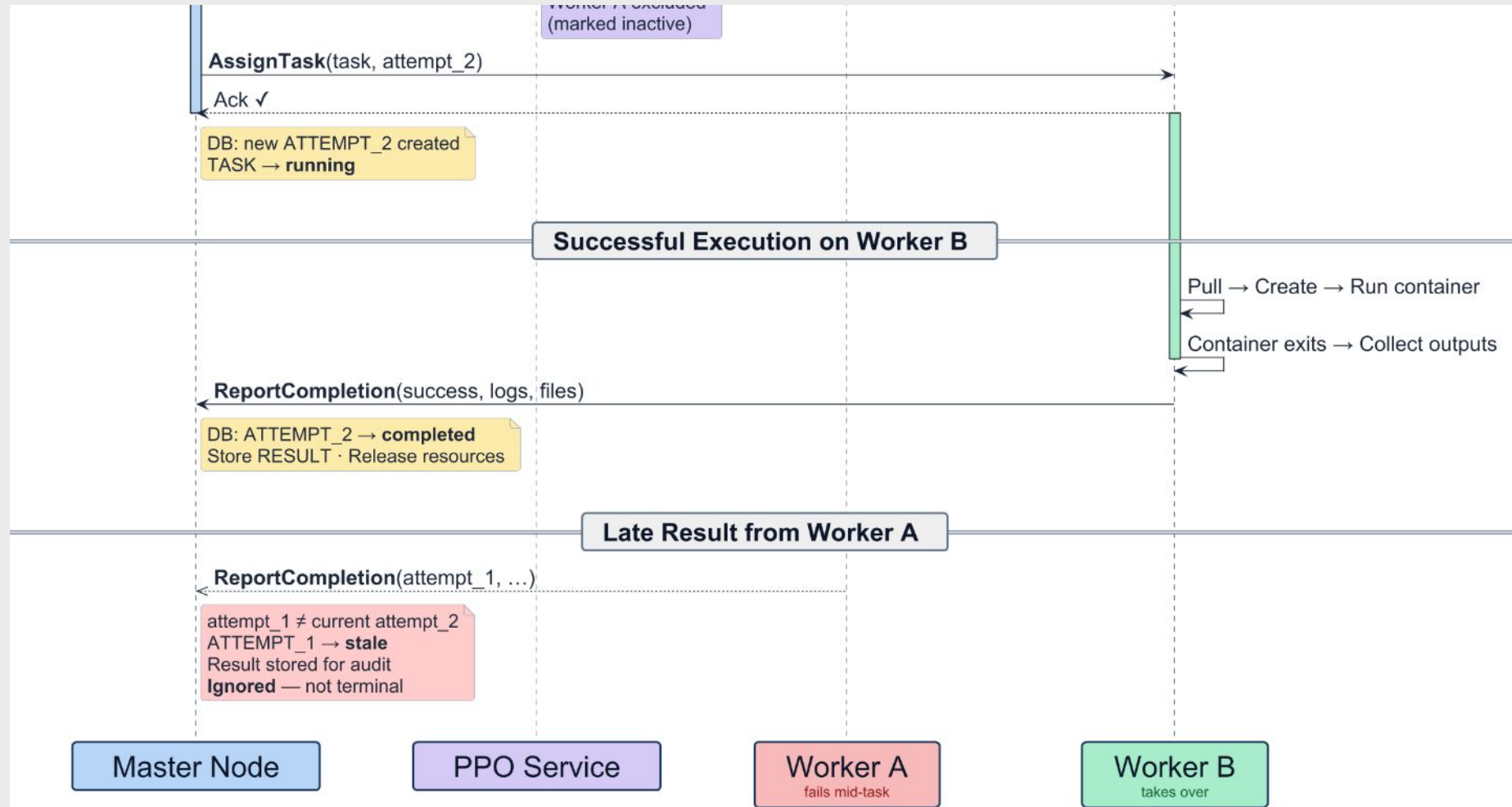
Task LifeCycle - Retry mechanism



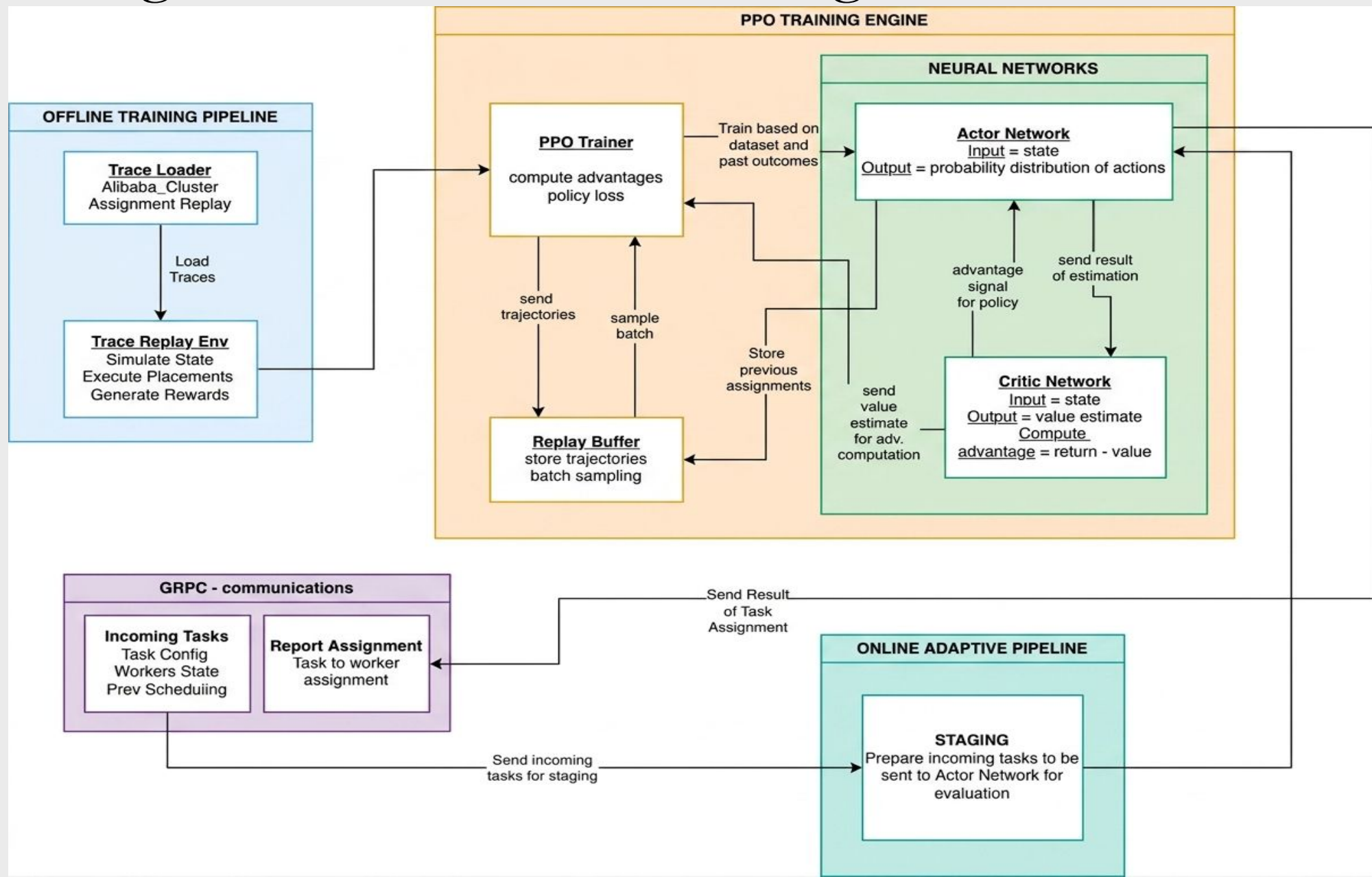
Task LifeCycle - Retry mechanism



Task LifeCycle - Retry mechanism



High-Level Architecture: Agentic Scheduler



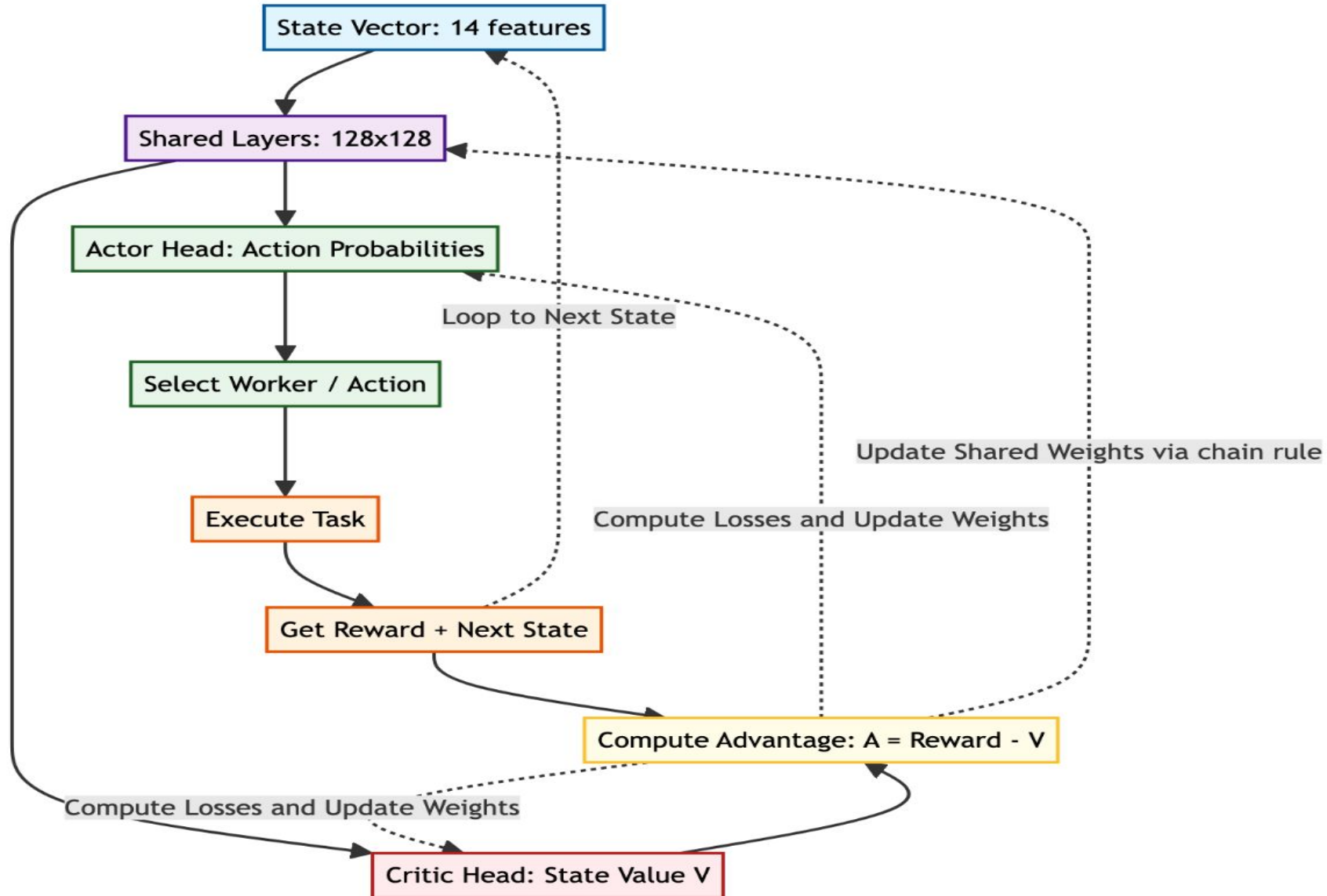
Methodology: Trace-Driven Training

- Trained using the Alibaba Cluster Trace v2018 (Production workload data).
- Replay Environment: Simulates sequential task arrivals, resource reservations, and completions.
- Lifecycle Resource Tracking: Matches actual simulated runtimes rather than inaccurate exponential decay.

State Representation & Action Masking

- Task Vector (5 features): CPU, Memory, Storage, SLA Multiplier, Task Type.
- Worker Vector (9 features): Available, Total, and Used ratios for CPU/Memory/Storage.
- Action Masking: Infeasible workers lacking required resources are mathematically hidden from the policy.

PPO Scheduler Design (Actor-Critic)

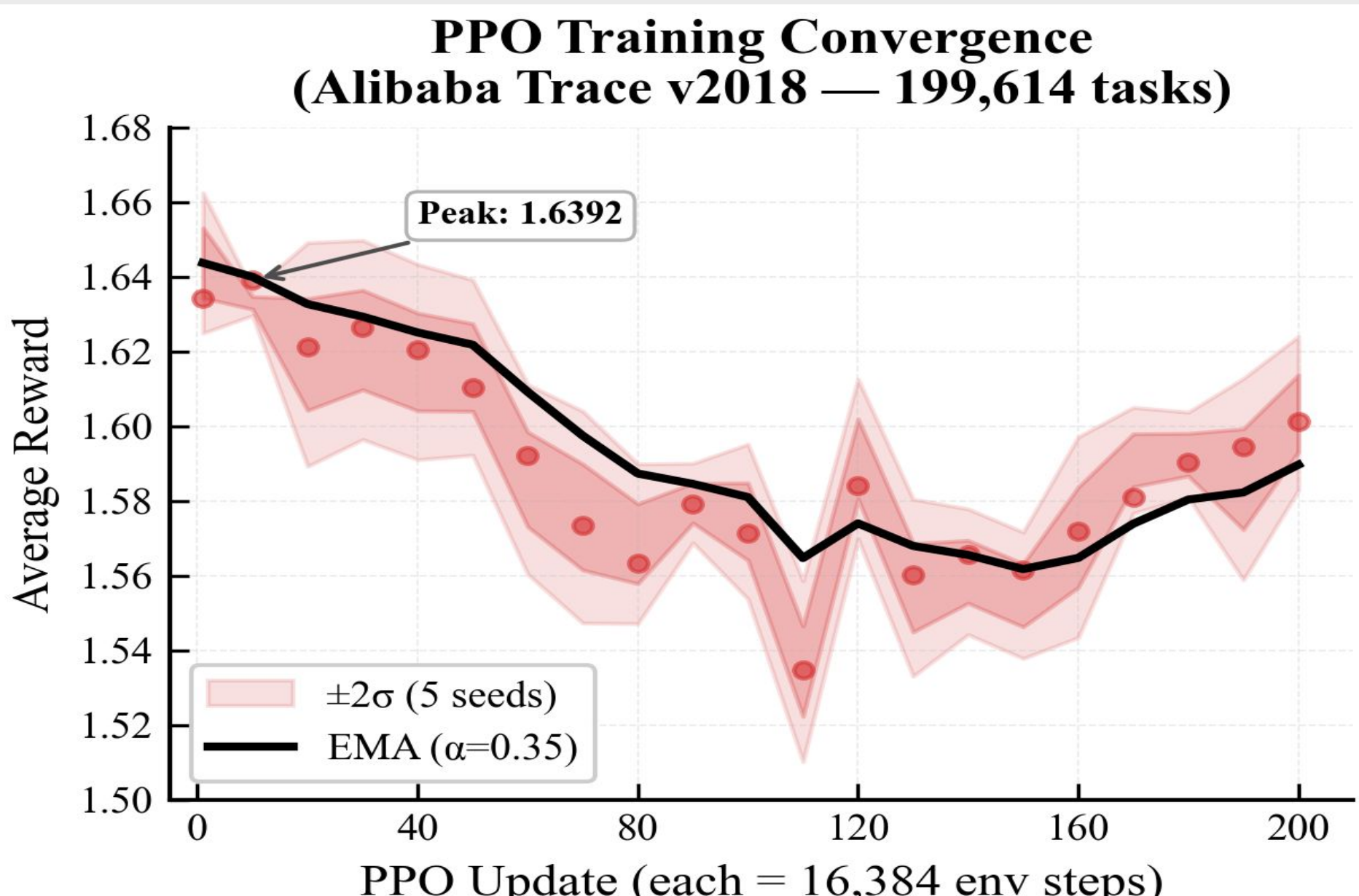


The Reward Function

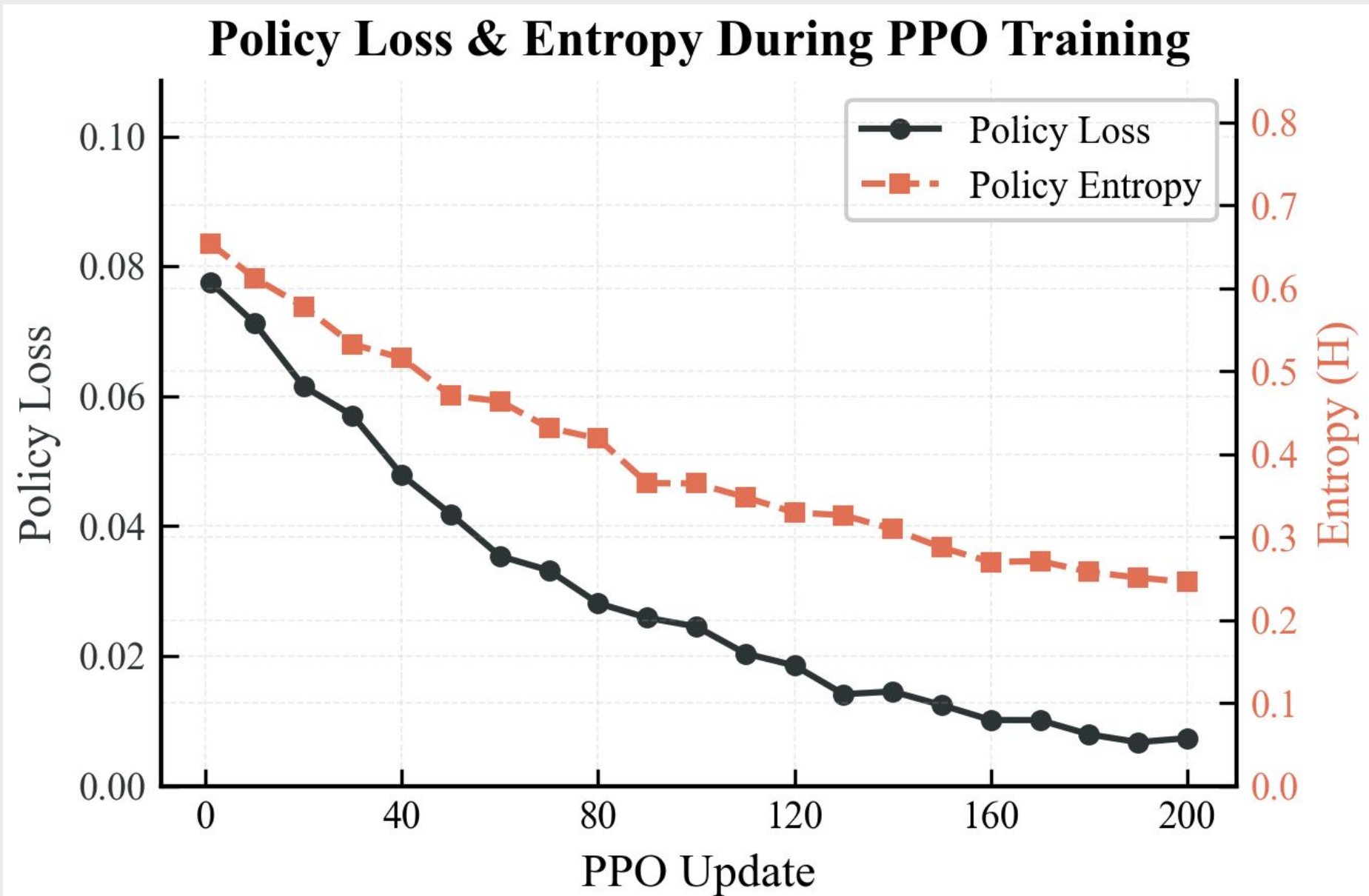
$$R = 1.4 + 0.25H - 0.35Q_p - 0.55T_p - 0.20I_p - 0.40\Delta I - R_q$$

- H is headroom bonus. It is rewarded by the selection of workers who have space vacant.
- Q_p is queue pressure. It punishes anticipated waiting.
- T_p is tail pressure. It penalizes likely SLA or tail-latency risk.
- I_p is imbalance penalty. It does not encourage overworking of already overworked workers.
- ΔI is change in imbalance. It punishes those behaviors that worsen cluster balance.
- R_q is requeue penalty. It discourages constantly dangerous placements.

Training Convergence

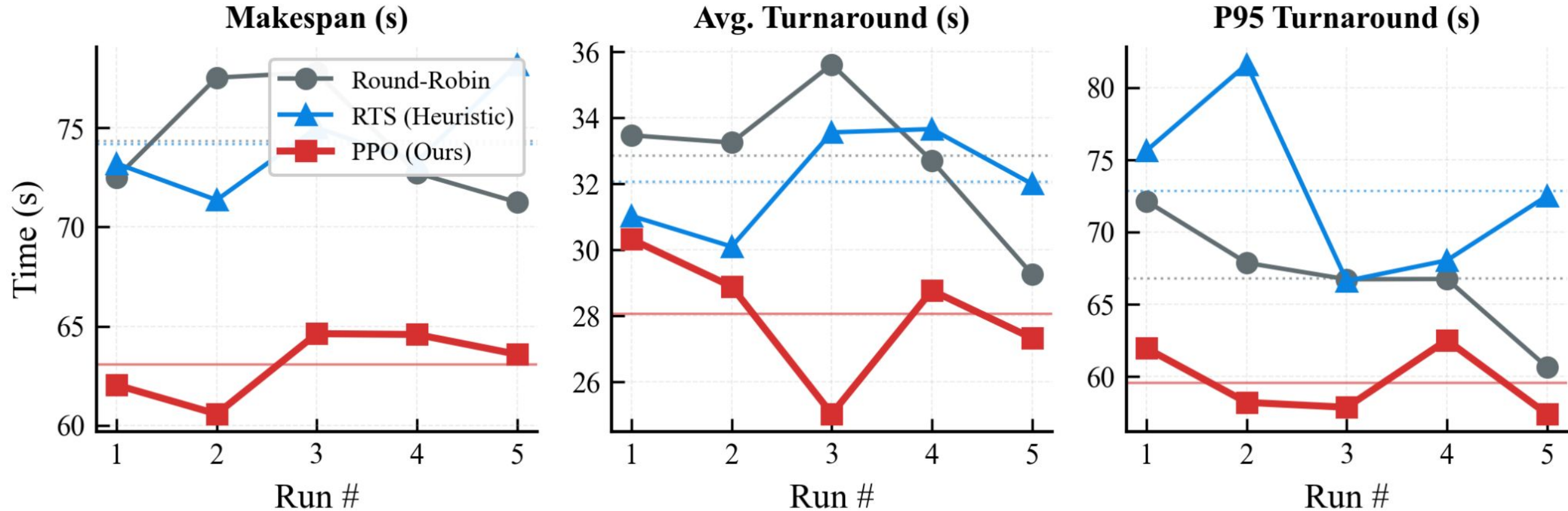


Training Convergence



Results: KPI Benchmarking

Scheduler Consistency Across Independent Runs (n=5)



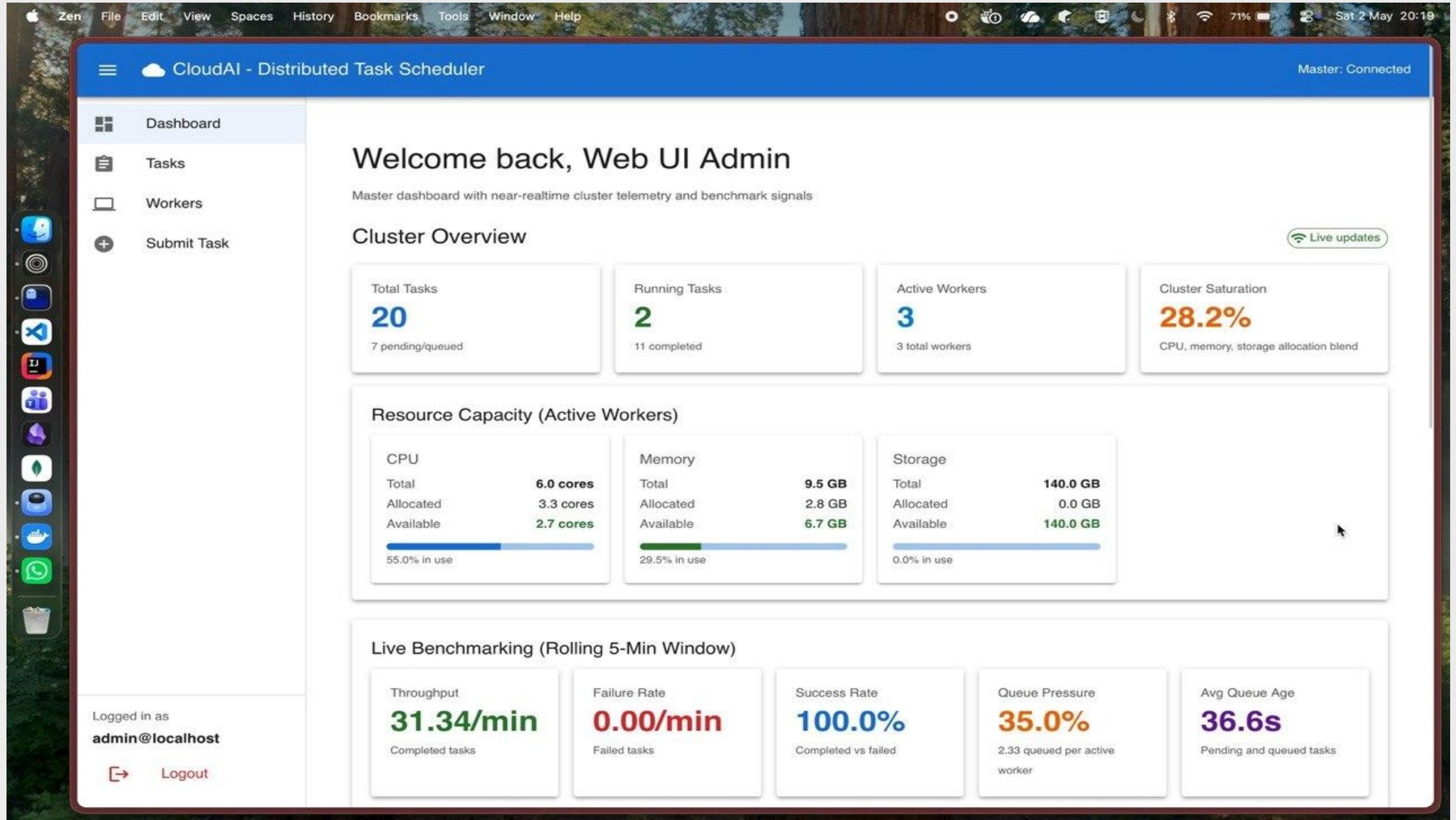
Pressure Scenarios & Cumulative Completion

Under bursty traffic:

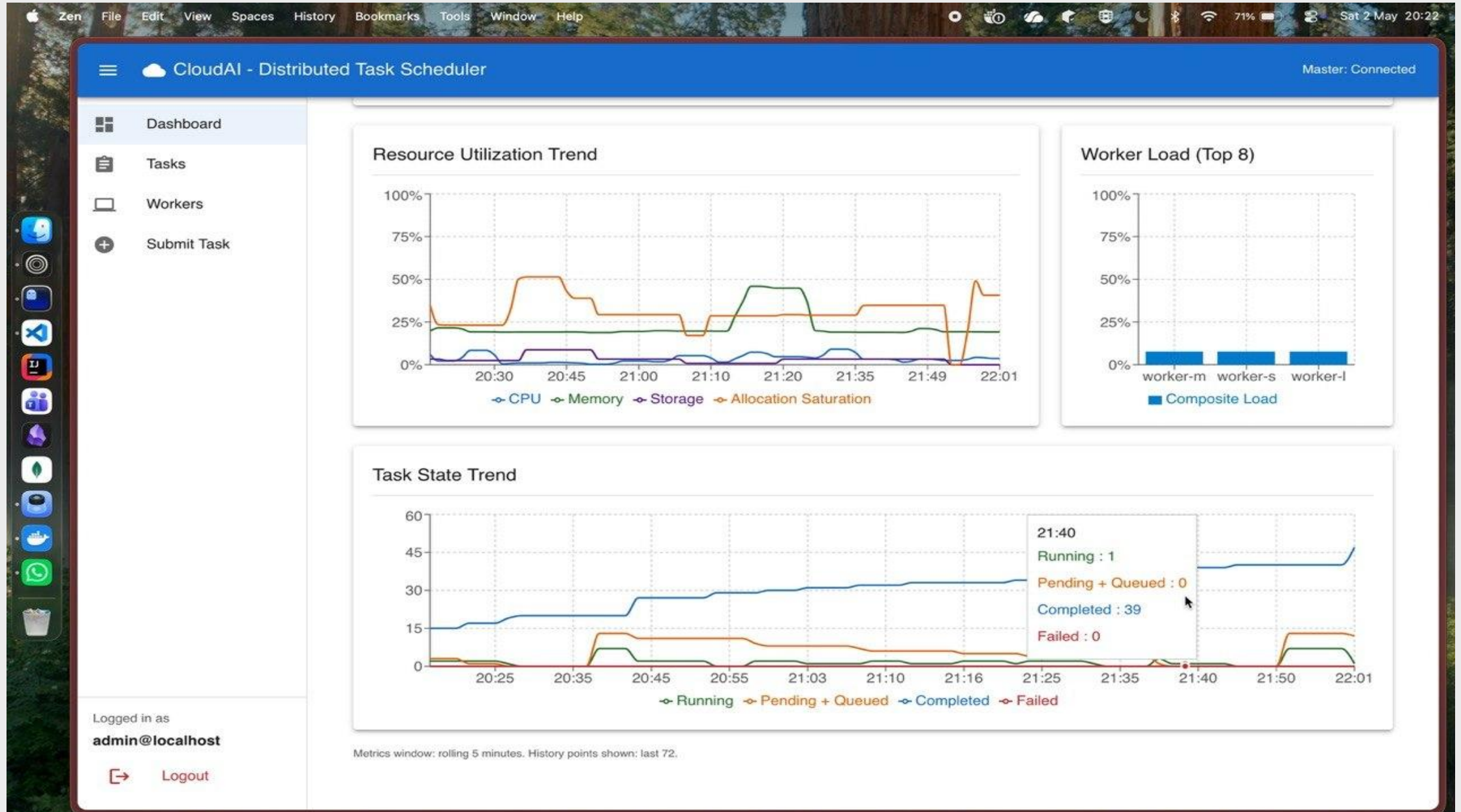
Comparison	Duration Improvement	Turnaround Improvement
PPO vs RR	8.7% faster	17.9% lower
PPO vs RTS	16.1% faster	16.9% lower

Scheduler	Tasks	Completed	Failed	Duration	Turnaround	P95
RR	20	20	0	70.06 s	33.80 s	69.00 s
RTS	20	20	0	76.20 s	33.40 s	74.00 s
PPO	20	20	0	63.95 s	27.75 s	62.00 s

Project Outcomes - Web UI



Project Outcomes - Live Plots



Project Outcomes - MasterNode CLI

```
tmux new -s report_old %1 ts %2 +
18:55 [12/164]

VITE v7.3.1 ready in 192 ms

→ Local: http://localhost:3001/
→ Network: use --host to expose

master>
master> help

Available commands:
  help                - Show this help message
  status              - Show cluster status
  workers             - List all registered workers
  stats <worker_id>   - Show detailed stats for a worker
  internal-state      - Dump complete in-memory state of all workers
  fix-resources       - Fix stale resource allocations
  list-tasks [status] - List all tasks (or filter by: queued/pending/running/completed/failed/cancelled)
  register <id> <ip:port> - Manually register a worker
  unregister <id>     - Unregister a worker
  task <docker_img> [-cpu_cores <num>] [-mem <gb>] [-storage <gb>] [-k <1.5-2.5>] [-type <task_type>]
                        - Submit task (scheduler selects worker)
  dispatch <worker_id> <docker_img> [options] - Dispatch task directly to specific worker (testing)
  monitor <task_id>   - Monitor live logs for a task
  cancel <task_id>    - Cancel a running task
  queue              - Show pending tasks in the queue
  benchmark [profile|all] - Run scheduler benchmark suite and generate report artifacts
  workload-submit <profile> [options] - Submit predefined workload to master at scheduled intervals
  files <user_id> [requesting_user] - List all files for a user
  task-files <task_id> <user_id> [requesting_user] - View files for a specific task
  download <task_id> <user_id> [requesting_user] [output_dir] - Download all task files
  exit/quit          - Shutdown master node

Task Types (-type flag):
  cpu-light          - Light CPU workloads
  cpu-heavy          - Heavy CPU workloads
  memory-heavy       - Memory-intensive workloads
  mixed              - Mixed CPU/memory/storage workloads

Examples:
  register worker-2 192.168.1.100:50052
  stats worker-1
  task docker.io/user/sample-task:latest
  task docker.io/user/sample-task:latest -cpu_cores 2.0 -mem 1.0 -storage 5.0
  task myapp:latest -cpu_cores 4 -mem 8 -k 1.8 -type cpu-heavy

[bte] 0:run- 1:copilot 2:zsh 3:defense 4:copilot 5:[tmux]*
"Thea" 18:55 04-May-26
```

Project Outcomes - Docker machines

```
tmux new -s report_old  ts  ~/personal/Projects
```

```
~/personal/Projects
>> docker ps
```

CONTAINER ID	IMAGE NAMES	COMMAND	CREATED	STATUS	PORTS
0f02d01c6b49	testbench-worker-small	"/app/workerNode"	18 hours ago	Up 18 hours (unhealthy)	0.0.0.0:19101→9101/tcp, [::]:19101→9101/tcp, 0.0.0.0:55052→50052/tcp, [::]:550
52→50052/tcp	testbench-worker-small-1	"/app/workerNode"	18 hours ago	Up 18 hours (unhealthy)	0.0.0.0:19102→9101/tcp, [::]:19102→9101/tcp, 0.0.0.0:55053→50052/tcp, [::]:550
6cb3b589b6a9	testbench-worker-medium	"/app/workerNode"	18 hours ago	Up 18 hours (unhealthy)	0.0.0.0:19102→9101/tcp, [::]:19102→9101/tcp, 0.0.0.0:55053→50052/tcp, [::]:550
53→50052/tcp	testbench-worker-medium-1	"/app/workerNode"	18 hours ago	Up 18 hours (unhealthy)	0.0.0.0:19103→9101/tcp, [::]:19103→9101/tcp, 0.0.0.0:55054→50052/tcp, [::]:550
2a882557519f	testbench-worker-large	"/app/workerNode"	18 hours ago	Up 18 hours (unhealthy)	0.0.0.0:19103→9101/tcp, [::]:19103→9101/tcp, 0.0.0.0:55054→50052/tcp, [::]:550
54→50052/tcp	testbench-worker-large-1	"/app/workerNode"	18 hours ago	Up 18 hours (unhealthy)	0.0.0.0:19103→9101/tcp, [::]:19103→9101/tcp, 0.0.0.0:55054→50052/tcp, [::]:550
ed994cd2b7f3	docker:27.3-dind	"dockerd-entrypoint...."	18 hours ago	Up 18 hours (healthy)	2375-2376/tcp
	testbench-worker-medium-dind-1	"dockerd-entrypoint...."	18 hours ago	Up 18 hours (healthy)	2375-2376/tcp
20470c6a78b6	docker:27.3-dind	"dockerd-entrypoint...."	18 hours ago	Up 18 hours (healthy)	2375-2376/tcp
	testbench-worker-large-dind-1	"dockerd-entrypoint...."	18 hours ago	Up 18 hours (healthy)	2375-2376/tcp
89fbc7d9fee5	docker:27.3-dind	"dockerd-entrypoint...."	18 hours ago	Up 18 hours (healthy)	2375-2376/tcp
	testbench-worker-small-dind-1	"dockerd-entrypoint...."	18 hours ago	Up 18 hours (healthy)	2375-2376/tcp
bd4376a8bc58	grafana/grafana:12.1.1	"/run.sh"	18 hours ago	Up 18 hours	0.0.0.0:3300→3000/tcp, [::]:3300→3000/tcp
	testbench-grafana-1	"/run.sh"	18 hours ago	Up 18 hours	0.0.0.0:3300→3000/tcp, [::]:3300→3000/tcp
17e165fc861c	prom/prometheus:v3.5.0	"/bin/prometheus --c...."	18 hours ago	Up 18 hours	0.0.0.0:9090→9090/tcp, [::]:9090→9090/tcp
	testbench-prometheus-1	"/bin/prometheus --c...."	18 hours ago	Up 18 hours	0.0.0.0:9090→9090/tcp, [::]:9090→9090/tcp
eff94390cfbd	mongo:7.0	"docker-entrypoint.s...."	18 hours ago	Up 18 hours (healthy)	127.0.0.1:27018→27017/tcp
	testbench-mongo-1	"docker-entrypoint.s...."	18 hours ago	Up 18 hours (healthy)	127.0.0.1:27018→27017/tcp
bae1baa165a7	mongo:7.0	"docker-entrypoint.s...."	41 hours ago	Up 41 hours (healthy)	127.0.0.1:27017→27017/tcp
	cloudai-mongo	"docker-entrypoint.s...."	41 hours ago	Up 41 hours (healthy)	127.0.0.1:27017→27017/tcp

```
~/personal/Projects
>>
```


Project Outcomes - DB & Worker Registry

The screenshot shows the MongoDB Compass interface for the `local_acc/cluster_db.WORKER_REGISTRY` collection. The left sidebar shows the database structure, including the `WORKER_REGISTRY` collection. The main panel displays two documents from the collection.

Document 1:

```
{
  "_id": ObjectId('69f60edc7f9654a1b5ede4cc'),
  "worker_id": "worker-small",
  "worker_ip": "127.0.0.1:55052",
  "total_cpu": 1,
  "total_memory": 1.5,
  "total_storage": 20,
  "allocated_cpu": 0,
  "allocated_memory": 0,
  "allocated_storage": 0,
  "available_cpu": 1,
  "available_memory": 1.5,
  "available_storage": 20,
  "is_active": true,
  "last_heartbeat": 1777814597,
  "registered_at": "2026-05-02T14:49:00.658+00:00",
  "updated_at": "2026-05-03T13:23:17.768+00:00"
}
```

Document 2:

```
{
  "_id": ObjectId('69f60edc7f9654a1b5ede4cd'),
  "worker_id": "worker-medium",
  "worker_ip": "127.0.0.1:55053",
  "total_cpu": 2,
  "total_memory": 3,
  "total_storage": 40,
  "allocated_cpu": 0,
  "allocated_memory": 0,
  "allocated_storage": 0,
  "available_cpu": 2,
  "available_memory": 3,
  "available_storage": 40,
  "is_active": true
}
```


Novelty in the project / Your contributions in the project

- Designed and built the Go master-worker framework from scratch.
- Engineered the clean scheduler integration boundary and gRPC communications.
- Developed the PPO training pipeline and trace replay environment.
- Implemented attempt-level recovery logic for fault tolerance.

Learning outcomes for you

- **Distributed Systems:** Learned the criticality of strict state handling, retries, and clean API boundaries.
- **Reinforcement Learning:** Realized the massive impact of reward shaping and safe deployment strategies.

Future Work - CRIU

- CRIU offers kernel-level support of freezing process state, dumping it to disk, and restoring it on demand.
- Benefits:
 - Container Migration
 - Task Preemption
 - Checkpointing failed workers



<https://github.com/checkpoint-restore/criu>26-02-23

Gantt chart

Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15	W16	W17
Problem study and literature review	X	X	X														
Go master-worker framework		X	X	X	X												
Worker execution, heartbeats, and persistence				X	X	X	X	X									
Baseline schedulers and scheduler interface						X	X	X	X								
PPO training pipeline and trace replay								X	X	X	X	X					
PPO integration with Go control plane											X	X	X				
Benchmark campaigns and result analysis													X	X	X	X	
Report writing and final polishing																	X

Who will benefit from your work

1. Companies — Better handling of burstier traffic
2. Developers — Automate scheduling, fewer manual deployments
3. Users — Faster, more reliable services

Bibliography

- Gu, Y., Liu, Z., Dai, S., Liu, C., Wang, Y., Wang, S., Theodoropoulos, G., & Cheng, L. (2025, January 2). Deep Reinforcement Learning for job scheduling and Resource Management in Cloud Computing: An Algorithm-Level Review. arXiv.org. <https://arxiv.org/abs/2501.01007>
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017, July 20). Proximal Policy optimization Algorithms. arXiv.org. <https://arxiv.org/abs/1707.06347>
- Alibaba, “Alibaba cluster data.” <https://github.com/alibaba/clusterdata>, 2018. Cluster traces released for cluster management research.
- <https://github.com/checkpoint-restore/criu>

Thank You!